US Patent & Trademark Office
Patent Public Search | Text View

United States Patent	12399756
Kind Code	B2
Date of Patent	August 26, 2025
Inventor(s)	Liu; Peng et al.

Method of optimizing dependent task offloading in internet of vehicles using deep reinforcement learning

Abstract

A method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning is provided, including the following steps: S**1**: constructing a vehicle system network; S**2**: constructing a task model for an application program; S**3**: constructing a task load model: calculating delay, energy consumption and incentive compensation for local computing, offloading to nearby vehicles and offloading to nearby RSUs three offloading methods based on the vehicle network system and the task model, respectively; S**4**: determining task priorities: first determining the priorities of each of subtasks according to allocation of predecessor nodes of the subtasks combining with dynamic network environment, and then scheduling according to a multi-queue algorithm; and S**5**: finding an optimal offloading strategy by using the deep reinforcement learning.

Inventors: **Liu; Peng (Hangzhou, CN), Zhang; Fei (Hangzhou, CN)**

Applicant: **HANGZHOU DIANZI UNIVERSITY** (Hangzhou, CN)

Family ID: **1000008781512**

Assignee: **HANGZHOU DIANZI UNIVERSITY (Hangzhou, CN)**

Appl. No.: **19/032177**

Filed: **January 20, 2025**

Prior Publication Data

Document Identifier	Publication Date
US 20250165311 A1	May. 22, 2025

Foreign Application Priority Data

CN	202310522700.6	May. 10, 2023

Related U.S. Application Data

continuation parent-doc WO PCT/CN2024/081842 20240315 PENDING child-doc US 19032177

Publication Classification

Int. Cl.: **G06F9/50** (20060101); **G07C5/00** (20060101)

U.S. Cl.:

CPC **G06F9/5094** (20130101); **G07C5/008** (20130101);

Field of Classification Search

USPC: None

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
11206221	12/2020	Guo	N/A	H04L 47/623
2021/0136142	12/2020	Dong	N/A	G06F 9/5094
2022/0126725	12/2021	Wang	N/A	B60L 53/63

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
114116047	12/2021	CN	N/A
116321298	12/2022	CN	N/A
116455903	12/2022	CN	N/A

2022027776	12/2021	WO	N/A

Primary Examiner: Wai; Eric C

Attorney, Agent or Firm: Birchwood IP

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS (1) This application is a continuation of PCT/CN2024/081842, filed on Mar. 15, 2024 and claims priority of Chinese Patent Application No. 202310522700.6, filed on May 10, 2023, the entire contents of which are incorporated herein by reference.

TECHNICAL FIELD

(1) The present disclosure relates to vehicle edge computing, and specifically relates to a method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning.

BACKGROUND

(2) With the dramatic growth in the number of vehicle users worldwide and the increasing demand for services from vehicle users, telematics networks have endured more business challenges such as high transmission bandwidth requirements of the network, high transmission reliability and strong data processing capabilities. Internet of vehicles (IoV) integrates traditional vehicle Ad Hoc Network and telematics, and thus can effectively improve vehicle service capability. In IOV, intelligent vehicles can execute a variety of applications, such as collision warning, autonomous driving, and auto-navigation. However, these applications require a large amount of computational and storage resources and have strong delay requirements. It is a challenging task to execute these applications on resource-limited vehicles under low delay constraints.

(3) In-vehicle edge computing that integrates Mobile Edge Computing (MEC) into IoV is a promising solution to effectively address the above problems. In-vehicle edge computing may improve vehicle quality of service by deploying computation of MEC servers and storage resources in the vicinity of vehicles. Compute-intensive and delay-sensitive tasks may be loaded onto the MEC servers for execution over a wireless network. Although in-vehicle edge computing may reduce the delay of task execution, the edge servers have limited computation and storage capacity and thus cannot guarantee load balancing. Therefore, an effective offloading strategy may reduce the processing delay of tasks and improve the quality of service for users.

(4) Current in-vehicle edge computing mainly study the computational optimization of individual tasks and lack consideration of the dependencies between tasks, and thus limits the application scope and effectiveness of in-vehicle edge computing technologies. In addition, considering the environmental protection factors, most vehicles are electric vehicles, and energy consumption is crucial for the use of electric vehicles. In practical applications, vehicles or infrastructures providing computing services usually require corresponding incentive compensation, but existing studies rarely focus on energy consumption in computing and corresponding incentive compensation. To address these issues, the present disclosure aims to minimize the energy consumption and the incentive compensation paid during computation while satisfying the demand for computation services of an application, and thereby improve the quality of service for the user. Specifically, the present invention proposes a system and method for optimizing the offloading of dependent tasks in the IOV using deep reinforcement learning, taking into account the dependency relationships between the tasks so as to ensure that the delay constraints are satisfied while minimizing the energy consumption in the computation process and the incentive compensation paid to the vehicle or infrastructure for the computation service. Specifically, the computational resources, energy costs and incentive compensation of vehicles and infrastructures are first mathematically modeled, and then corresponding task scheduling and offloading strategies are developed using deep reinforcement learning techniques and incorporating the dependencies between tasks. This ensures that the delay constraints are satisfied while minimizing the energy consumption in the computation process and the incentive compensation paid to the vehicles or infrastructures providing the computation services.

(5) In summary, the present disclosure proposes a novel in-vehicle edge computing technology that may better solve the computational offloading problem of dependencies between tasks, energy costs and incentive compensation, and improve the efficiency and quality of computing services.

SUMMARY

(6) An objective of the present disclosure to provide a method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning, so as to overcome the shortcomings of the prior art.

(7) In order to achieve the above objective, the present disclosure provides a method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning, comprising the following steps: S**1**: constructing a vehicle system network, wherein the vehicle system network comprises MEC servers, Road Side Units (RSUs) and vehicular user sides connected in sequence; S**2**: constructing a task model for an application program, wherein tasks with temporal dependencies and data dependencies are modeled as a DAG task model; S**3**: constructing a task load model: calculating delay, energy consumption and incentive compensation for local computing, offloading to nearby vehicles and offloading to nearby RSUs three offloading methods based on the vehicle network system and the task model, respectively; S**4**: determining task priorities: first determining the priorities of each of subtasks according to allocation of predecessor nodes of the subtasks combining with dynamic network environment, and then scheduling according to a multi-queue algorithm; and S**5**: finding an optimal offloading strategy by using the deep reinforcement learning: each of the subtasks selects a corresponding execution device to complete the task offloading, so as to minimize energy consumption and paid incentive compensation under constraints of delay, and thereby improve quality of service for users.

(8) Compared to the prior art, the present disclosure has the following technical effects.

(9) This disclosure is applicable to V2I and V2V collaborative offloading scenarios in an IoV environment. The method decouples the nonlinear integer programming problem into two subproblems: the associated subtask scheduling problem and the task offloading problem. Combining the allocation of predecessor tasks of the subtasks and the dynamic network environment, the method prioritizes the set of ready subtasks and then solves the offloading problem using deep reinforcement learning based on value functions and policy functions. The method optimizes the task offloading problem, and minimizes the energy consumption during computation and the incentive compensation paid to the vehicles or infrastructures with the computation service, while ensuring task dependency and the constraints of delay. The present disclosure does not require excessive a priori knowledge, has good reusability in similar application scenarios, and thus is of high practical value.

(10) The present disclosure provides a method for dependent task offloading in V2I and V2V collaborative edge computing based on deep reinforcement learning, effectively minimizes the weighted sum of the energy consumption and the incentive compensation paid to the client-side vehicle under the constraints of delay, and thereby improve the user experience. The process of the present disclosure is standardized, and is easy to operate. Moreover, the present disclosure also has strong practical value and reusability, and has a wide range of application prospects in similar application scenarios.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) FIG. **1** is a schematic diagram of the V2I and V2V collaborative offloading model of the embodiment in this disclosure.

(2) FIG. **2** shows the schematic diagram of the deep reinforcement learning algorithm of the embodiment in this disclosure.

(3) FIG. **3** is a flowchart based on deep reinforcement learning algorithm in an IoV scenario of the embodiment in this disclosure.

DETAILED DESCRIPTION OF THE EMBODIMENTS

(4) The present disclosure is further described below in connection with the drawings and specific embodiments so that those skilled in the art may better understand the present disclosure and be able to allow implementation, but the cited implementations are not intended to be a limitation of the present disclosure.

(5) An agent takes an action in the current state to obtain a reward from the environment, and then enters the next state. Through continuous interaction, the agent may gradually learn the optimal offloading strategy. In this process, the agent constantly updates its value function to better estimate the value of each action. At the same time, the agent also continuously adjusts strategy in order to better explore the search space and find the optimal offloading decision.

(6) In order to minimize the weighted sum of energy consumption and incentive compensation while satisfying delay constraints in an IoV edge computing scenario combining Vehicle-To-Infrastructure (V2I) offloading and Vehicle-To-Vehicle (V2V) offloading, the present disclosure provides a method for dependent task offloading based on deep reinforcement learning.

(7) The edge computing scenario described in the present disclosure refers to vehicle nodes in an IoV network communication system combining V2I offloading and V2V offloading. These vehicle nodes may carry out V2I communication with edge Mobile Edge Computing (MEC) servers and may carry out V2V communication with other vehicle nodes. In this scenario, the vehicle that generates the application program is called a client-side vehicle and the vehicles that provide computational services to the application program are called server-side vehicles. The application program includes a series of indivisible subtasks with temporal dependencies and data dependencies that can be represented using a directed acyclic graph (DAG). These subtasks can be executed locally, transferred to server-side vehicles for execution, or uploaded to Rode side Units (RSUs) next to the highway for execution. In order to achieve minimizing the weighted sum of energy consumption and incentive compensation in this scenario, this disclosure provides a method for dependent task offloading based on deep reinforcement learning, which requires very little a priori knowledge and simple interactions with the environment to gain learning experience and obtains a near-optimal solution for minimizing the weighted sum of energy consumption and incentive compensation.

(8) As shown in FIG. **3**, the present disclosure provides a method of optimizing dependent task offloading in an IoV using deep reinforcement learning, including the following steps.

(9) S**1**: Constructing a vehicle system network.

(10) The proposed vehicle system network includes MEC servers, RSUs and vehicular user sides connected in sequence. A plurality of vehicle user sides is connected to each other via V2V transmission links to realize inter-vehicle communication and collaborative computing. Vehicle user sides and RSUs are connected via V2I transmission links to realize the communication between vehicles and the network. The MEC servers, as edge computing resource providers providing resources such as computation, storage and network, may provide edge computing services for the vehicle user sides. RSUs, as the infrastructures of the network, provide information such as network connection and road conditions and provide data and collaborative computation support for the vehicle user sides. The whole vehicle system network constitutes a distributed edge computing platform, and realizes cooperative computing and communication between vehicles and between vehicles and networks.

(11) As shown in FIG. **1**, in this scenario, according to roles in task processing, vehicles can be divided into a client-side vehicle and server-side vehicles. The data transmission links include V2V transmission links for task data, the upload links for task data to RSUs, the connection links between RSUs and servers, and the connection links between RSUs and RSUs. For the client-side vehicle, its tasks can be processed by the following three offloading methods: 1) Local computing: the client-side vehicle processes tasks by itself; 2) Offloading to nearby vehicles: the client-side vehicle uploads the tasks to neighboring server-side vehicles; 3) Offloading to nearby RSUs: the client-side vehicle uploads tasks to nearby RSUs.

(12) S**2**: Constructing a task model for the application program:

(13) Tasks with temporal dependencies and data dependencies are modeled as a DAG task model, and the DAG task is modeled as G=(V,E), where the vertex set V={ϕ.sub.i|1≤i≤N}, N denotes number of subtasks in V, the triad is defined as ϕ.sub.i=(w.sub.i, d.sub.i, τ.sub.i), w.sub.i, d.sub.i, τ.sub.i denote the computational workload, the data size, and the maximum completion time tolerance delay of the task ϕ.sub.i, respectively, E denotes the set of edges, and each edge e.sub.ϕ.sub.i.sub.,ϕ.sub.jϵE is a directed edge denoting the priority of the execution between them, indicating that the subtask ϕ.sub.i needs to be completed before ϕ.sub.j (which is equivalent to the fact that the output of ϕ.sub.i is used as a part of the input of ϕ.sub.j). If an edge exists from ϕ.sub.i to ϕ.sub.j, ϕ.sub.i is call as an immediate predecessor of ϕ.sub.j; otherwise, ϕ.sub.j is called an immediate successor of ϕ.sub.i. All predecessors need to be completed before their successors. A subtask without any predecessor node is a start node, also known as a source task, denoted as ϕ.sub.entry; a subtask without any successor node is an exit node, denoted as ϕ.sub.exit. With the DAG task model, the temporal dependencies and data dependencies between tasks can be clearly represented, which provides a basis for task scheduling.

(14) S**3**: Constructing the task load model: based on the vehicle network system and the task model, calculating the delay, energy consumption and incentive compensation for the three offloading methods, namely, local computing, offloading to nearby vehicles, and offloading to nearby RSUs, respectively.

(15) S**301**: Interdependent subtasks executed at different vehicle nodes require data transfer. Specifically, it takes some time to transfer the computation results of subtask ϕ.sub.j executed on vehicle node m to subtask ϕ.sub.i executed on vehicle node n, and this data transfer time is defined as follows:

(16) $$TT_{j,i}(m,n)=\begin{cases}\frac{e_{j,i}}{\tau_{mn}}, & e_{mn}\in E^{RSU_i}\\ \min\left(\frac{e_{j,i}}{\tau_{mk}}+\frac{e_{j,i}}{\tau_{kn}}\right), & e_{mk}\in E^{RSU_i}\ \text{and}\ e_{kn}\in E^{RSU_i}\ \text{and}\ e_{mn}\ .\text{Math.}\ E^{RSU_i}\end{cases}, \quad (1)$$

(17) where k, m and n represent the labeling of the vehicle nodes respectively, ϕ.sub.j is the predecessor node of subtask of ϕ.sub.i, τ.sub.mn represents the transmission bandwidth between the vehicle node m to the vehicle node n, e.sub.ϕ.sub.j.sub.ϕ.sub.i denotes the data size of the dependent data of the subtasks ϕ.sub.i to ϕ.sub.j, e.sub.mn denotes whether or not the vehicle nodes m and n are within the communication range, and E.sup.RSU.sup.i denotes the set of edges established by vehicles through single-hop V2V communication within the communication range of road side unit RSU.sub.i.

(18) S**302**: Calculating the time to execute the task locally.

(19) Specifically, for the case of executing the task locally, the completion time of the subtask ϕ.sub.i includes the waiting time t.sub.i.sup.local,wait and the compution time t.sub.i.sup.local,compution.

(20) $$t_i^{\text{local, complete}}=t_i^{\text{local, wait}}+t_i^{\text{local, computation}}, \quad (2)\ t_i^{\text{local, computation}}=\frac{w_i}{f_c}, \quad (3)\ t_i^{\text{local, wait}}=\max\{\text{avail}(V_c),\max(TT_{j,i}(m,V_c))\}, \quad (4)$$

(21) where V.sub.c denotes the client-side vehicle, avail(V.sub.c) denotes the time when the client-side vehicle finishes all the scheduled tasks, w.sub.i denotes the amount of computation required by the subtask ϕ.sub.i, and f.sub.c denotes the computational power of the client-side vehicle.

(22) For the case of executing tasks on RSU.sub.j, the completion time of subtask ϕ.sub.i includes upload time t.sub.i.sup.RSU.sup.j.sup.,upload, waiting time t.sub.i.sup.RSU.sup.j.sup.,wait and compution time t.sub.i.sup.RSU.sup.j.sup.,compution

(23) $t_i^{\mathrm{RSU}_j,\,\mathrm{complete}} = t_i^{\mathrm{RSU}_j,\,\mathrm{upload}} + t_i^{\mathrm{RSU}_j,\,\mathrm{wait}} + t_i^{\mathrm{RSU}_j,\,\mathrm{compution}}$, (5) $t_i^{\mathrm{RSU}_j,\,\mathrm{upload}} = \frac{d_i}{r_{V_c,\mathrm{RSU}_j}}$, (6)

$t_i^{\mathrm{RSU}_j,\,\mathrm{wait}} = \max\{\mathrm{avail}(RSU_j), \max(TT_{j,\,i}(m, RSU_j))\}$, (7) $t_i^{\mathrm{RSU}_j,\,\mathrm{computation}} = \frac{w_i}{f_{\mathrm{RSU}_j}}$, (8)

(24) where r.sub.V.sub.c.sub.,RSU.sub.j denotes the transmission rate between the client-side vehicle and the specified RSU.sub.j, d.sub.i denotes the data size of the subtask ϕ.sub.i, and f.sub.RSU.sub.j denotes the computational power of the road side unit RSU.sub.j.

(25) Similarly, the time to execute task on the server-side vehicles includes transmission time t.sub.i.sup.V.sup.k.sup.,tans, waiting time t.sub.i.sup.V.sup.k.sup.,wait and compution time t.sub.i.sup.V.sup.k.sup.,compution.

(26) $t_i^{V_k,\,\mathrm{complete}} = t_i^{V_k,\,\mathrm{trans}} + t_i^{V_k,\,\mathrm{wait}} + t_i^{V_k,\,\mathrm{compution}}$, (9) $t_i^{V_k,\,\mathrm{trans}} = \frac{d_i}{r_{V_c,V_k}}$, (10) $t_i^{V_k,\,\mathrm{wait}} = \max\{\mathrm{avail}(V_k), \max(TT_{j,\,i}(m, V_k))\}$, (11)

$t_i^{V_k,\,\mathrm{compution}} = \frac{w_i}{f_k}$, (12)

(27) where r.sub.V.sub.c.sub.,V.sub.k denotes the transmission rate between the client-side vehicle V.sub.c to the specified server-side vehicle V.sub.k, and f.sub.k denotes the computational power of the server-side vehicle V.sub.k.

(28) S**303**: Calculating the energy consumption for executing tasks locally:

(29) $e_i^{\mathrm{local}} = (t_i^{\mathrm{local,\,compution}})^{-2}(w_i)^3$, (13) where ξ is a coefficient related to the architecture of the computer chip, and there is no incentive compensation because it is executed locally; calculating the energy consumption for executing the task at the RSUs:

(30) $e_i^{\mathrm{RSU}_j} = p_{V_c}\, t_i^{\mathrm{RSU}_j,\,\mathrm{upload}}$, (14) where p denotes the upload power, so the incentive compensation to be paid to the road side unit RSU.sub.j for processing subtask ϕ.sub.i is as follows:

(31) $C_{\mathrm{RSU}_j,\,i} = p_j t_i^{\mathrm{RSU}_j,\,\mathrm{compution}}$, (15) where p.sub.j denotes the fee charged by the road side unit RSU.sub.j per unit time (s); calculating the energy consumption for executing the task on the server-side vehicles:

(32) $e_i^{V_k} = p_{V_c}\, t_i^{V_k,\,\mathrm{trans}}$, (16) where the incentive compensation to be paid to the server-side vehicle k for processing the subtask ϕ.sub.i is as follows:

C.sub.V.sub.k.sub.,ϕ.sub.i=*w*.sub.iϕ.sub.k (17), where ρ.sub.k denotes the fee charged by the server-side vehicle k per unit of resource (1G clocks).

(33) S**304**: The time t.sub.i, the energy consumption e.sub.i and the cost c.sub.i to be paid for executing a subtask ϕ.sub.i are:

(34) $t_i = x_i^{\mathrm{local}} t_i^{\mathrm{local,\,complete}} + x_i^{\mathrm{RSU}_j} t_i^{\mathrm{RSU}_j,\,\mathrm{complete}} + x_i^{V_k} t_i^{V_k,\,\mathrm{complete}}$, (18) $e_i = x_i^{\mathrm{local}} e_i^{\mathrm{local}} + x_i^{\mathrm{RSU}_j} e_i^{\mathrm{RSU}_j} + x_i^{V_k} e_i^{V_k}$, (19)

$c_i = x_i^{\mathrm{RSU}_j} C_{\mathrm{RSU}_j,\,i} + x_i^{V_k} C_{V_k,\,i}$, (20)

(35) where x.sub.i.sup.local, x.sub.i.sup.RSU.sup.j and x.sub.i.sup.V.sup.k denote the offloading decisions of the subtasks, and the values of x.sub.i.sup.local, x.sub.i.sup.RSU.sup.j and x.sub.i.sup.V.sup.k take all 0 or 1, and only one of the three can take 1.

(36) Total incentive compensation cost, i.e., the incentive compensation to be paid to the client-side vehicle to complete all tasks, is denoted as:

(37) 0 $C(X) = \sum_{i=1}^{N} c_i$, (21)

(38) Total energy consumption cost, i.e., the energy consumption of the client-side vehicle to complete all tasks, denoted as:

(39) $E(X) = \sum_{i=1}^{N} e_i$, (22)

(40) where X denotes the offloading decision vector and N denotes the number of tasks.

(41) These formulas describe the relationship between task offloading decisions, execution time, energy consumption, and incentive compensation, and can be used to optimize the task offloading scheme to minimize the total energy consumption and total incentive compensation.

(42) S**305**: In order to make the optimal offloading decision, an optimization problem considering the energy consumption and incentive compensation for completing the tasks is created:

(43)

$$\min_{X}: (X) = C(X) + E(X) s.t. C1: x_i^{\mathrm{local}} + x_i^{\mathrm{RSU}_j} + x_i^{V_k} = 1, x_i^{\mathrm{local}} \in \{0,1\} x_i^{\mathrm{RSU}_j} \in \{0,1\} x_i^{V_k} \in \{0,1\}, x_i^{\mathrm{local}} \in \{0,1\} x_i^{\mathrm{RSU}_j} \in \{0,1\} x_i^{V_k} \in \{0,1\}, C$$

(44) where C(X) denotes the total incentive compensation cost, E(X) denotes the total computational cost, C1 denotes that each subtask has to be offloaded to a certain device, C2 and C3 denote that if a subtask is offloaded to server-side vehicles or RSUs, completion time cannot exceed the maximum allowable time thresholds l.sub.c,k or l.sub.c,RSU.sub.j, and C4 denotes that the weighted sum of energy consumption and incentive compensation is 1, and both must be positive number.

(45) The optimization problem may be decomposed into two subproblems: task priority determination and offloading decision. First, the task priority determination is to determine the execution order of each subtask in order to satisfy the constraints of delay and task dependency. Then, the task scheduling order is used for offloading decisions, the deep reinforcement learning is used to decide whether each subtask is executed locally, offloaded to the RSUs, or offloaded to the server-side vehicles to minimize the weighted sum of energy consumption and incentive compensation.

(46) S**4**: Determining task priority: first determining the priority of each subtask according to the allocation of the predecessor node of the subtask combining with the dynamic network environment, and then scheduling according to the multi-queue algorithm.

(47) S**401**: Starting subtasks, assuming that Rank.sup.D(ϕ.sub.ready)=0, determining the priority of the successor subtasks based on the scheduling arrangement of the predecessor subtasks, determining the ranking of the existing subtasks ϕ.sub.1∈ϕ.sub.ready, and computing the value of the dynamic down-ranking as follows:

(48) $\mathrm{Rank}^D(\quad_i) = \max_{\quad_j \in \mathrm{pred}(\quad_i)} \{\mathrm{Rank}^D(\quad_j) + \overline{\mathrm{TT}}^{\,D}_{\quad_j,\,\quad_i}\} + \overline{\mathrm{CT}}^{\,D}_{\quad_i}$. (24) where pred(ϕ.sub.1) is the set of immediate predecessor nodes of subtask ϕ.sub.i, CT.sub.ϕ.sub.i.sup.D is the dynamic average compution time, which may be affected by the allocation of ϕ.sub.j∈pred(ϕ.sub.i), C.sup.VC(ϕ.sub.i) denotes the set of candidate server-side vehicles or infrastructures that may execute subtask ϕ.sub.i, and ϕ.sub.ready denotes the set of ready subtasks and is defined as follows:

(49) $\overline{CT}^{\,D}_{\quad_i} = \frac{\sum_{m \in C^{\mathrm{VC}}(\quad_i)} CT(\quad_i, m)}{\left|C^{\mathrm{VC}}(\quad_i)\right|}$ $\quad_i \in \quad_{\mathrm{ready}}$, (25) moreover, TT.sub.ϕ.sub.j.sub.,ϕ.sub.i.sup.D represents the dynamic average transmission time, which is also affected by the allocation of ϕ.sub.j∈pred(ϕ.sub.i), and is defined as follows:

(50) $\overline{TT}^{\,D}_{\quad_j,\,\quad_i} = \frac{\sum_{m \in C^{\mathrm{VC}}(\quad_i)} TT_{\quad_j,\,\quad_i}(n, m)}{\left|C^{\mathrm{VC}}(\quad_i)\right|}$ $\quad_i \in \quad_{\mathrm{ready}}$ $\quad_j \in \mathrm{pred}(\quad_i)$. (26)

(51) S**402**: Task scheduling depends not only on priority but also on dependencies between tasks. A subtask can only be executed when all its predecessor subtasks are completed. Therefore, it is necessary to maintain a queue of ready subtasks, Q.sup.ready, and a queue of executing subtasks, Q.sup.exe, along with a queue of computed subtasks, Q.sup.finished, and a set of all subtasks in the application program, T.sup.task.

(52) Initially, ϕ.sub.entry subtasks are placed into Q.sup.ready from the set T.sup.task, the tasks in Q.sup.ready are sorted in ascending order

according to the subtask priority determination formula, the queue head element is removed to place the tasks into the queue Q.sup.exe of executing subtasks, and if there are completed tasks in queue Q.sup.exe, the completed tasks are placed into the Q.sup.finished queue. This may generate new ready subtasks, and if all predecessor node subtasks of a subtask ϕ.sub.i in the set T.sup.task are in the queue Q.sup.finished, the subtask ϕ.sub.i becomes a ready subtask, which is placed in the queue Q.sup.ready, and the above steps are repeated until the set T.sup.task is empty. In the whole process of subtask priority determination, the ready subtasks are dynamically determined according to the execution results of the subtask predecessor, and thus the priority of the ready subtasks is determined.

(53) S**5**: Finding the optimal offloading strategy by using deep reinforcement learning, each subtask selects the corresponding execution device to complete the task offloading, so as to minimize energy consumption and incentive compensation under constraints of delay, thereby improving the quality of service for users, as shown in FIG. **2**.

(54) In deep reinforcement learning methods, the design of actions, states, and reward values plays a crucial role. In order to adapt to the system environment and enable the agent to output the optimal action based on the current environmental state. The state space, action space, and reward function are designed as follows:

(55) State space S: the state is represented in vector form, indicating the state of the client-side vehicle, including task information, location, available computing resources, and connection time. State space S.sub.t={T, N}, where T and N respectively represent the set of candidate vehicles and the set of ready subtasks available within the communication range of the road side unit RSU.sub.j,

(56) $T = (V^{\mathrm{RSU}_j}), N = (w_i, \ _i, \mathrm{succ}(\ _i), \mathrm{pred}(\ _i))$, (27)

(57) Action space A: the action is a decision made by the requester through the offloading strategy π: A←S, and has a direct impact on the external environment. The action at time t is denoted as A.sub.t={x.sub.i.sup.local,x.sub.i.sup.RSU.sup.j,x.sup.V.sup.k}, x_i{circumflex over ( )}(V_k)}, which is a one-hot encoding. The value of each element can be 0 or 1.

(58) Reward function R: intuitively speaking, the Reward R←(S,A), the feedback obtained by the agent during the offloading process, directly determines the strategy. The optimization goal is to minimize energy consumption and overhead costs, but in reality, each decision can only serve one subtask. Therefore, the cost of one subtask m at time t is represented as R.sub.t(s, a)=−(e.sub.i+c.sub.i).

(59) $$R_t(s,a) = \begin{cases} -e_i^{\mathrm{local}}, & x_i^{\mathrm{local}} = 1 \\ -(e_i^{\mathrm{RSU}_j} + C_{\mathrm{RSU},j,\ _i}), & x_i^{\mathrm{RSU}_j} = 1 \\ -(e_i^{V_k} + C_{V_k,\ _i}), & x_i^{V_k} = 1 \end{cases}, \quad (28)$$

(60) S**501**: Initializing the experience replay buffer and defining the maximum number of executions and initial episodes for training and evaluation.

(61) S**502**: After initialization, initializing the parameters μ and θ of the current actor and current critic. This is because before starting training, the network parameters need to be initialized to some random values in order to avoid getting stuck in local optima during the learning process.

(62) S**503**: Initializing the target actor's parameters μ′←μ and the target critic parameters θ′←θ separately. These networks are used during the algorithm training process to calculate TD errors and update the parameters of the current network.

(63) S**504**: Obtaining the initial state s.sub.t of the environment is an important step in the algorithm. At the same time, it is necessary to prepare the initial random noise n.sub.t for action exploration. Through such exploration, more pairs of states and actions can be discovered, and the network can better converge to the optimal solution.

(64) S**505**: From time t=1 until t=n (where n represents the number of indivisible subtasks in the application program). a.sub.t is obtained based on the output of the Actor-C network of the current network and the random noise n.sub.t:

(65) $a_t = \ (s_t) + n_t$, (29)

(66) S**506**: Executing action a.sub.t, receiving reward r.sub.t, and the environment state changing to s.sub.t+1.

(67) S**507**: Storing (s.sub.t, a.sub.t, r.sub.t, s.sub.t+1) in the experience replay buffer, utilizing the historical data in the experience replay buffer for offline training and improving the stability and robustness of the model.

(68) S**508**: Sampling N tuples {(s.sub.t, a.sub.t, r.sub.t, s.sub.t+1)} from the experience replay buffer for training the target network and updating the current network, where usually, random sampling is used to ensure the randomness of the sample.

(69) S**509**: Using the target network to calculate the target value for each tuple y.sub.t, where y.sub.t is a value related to the current state s.sub.t and action a.sub.t, y.sub.t includes the current reward r.sub.t and the target predicted value of the next state s.sub.t+1. Specifically, the target value is calculated using the following formula:

(70) $y_t = r_t + \gamma \vartheta \cdot (s_{t+1}, \ \cdot (s_{t+1}))$, (30)

(71) where γ is the discount factor, ϑ.sub.θ′ is the target critic, φ.sub.μ′ is the target actor, and these network parameters are initialized in S**503**.

(72) S**510**: updating the parameter θ of the current critic by minimizing target loss L(θ), where L(θ) is calculated from the mean squared error (MSE) loss function as follows:

(73) 0 $L(\ ) = \frac{1}{N}$ .Math.$_{i=1}^{N}$ $(y_i - \ (s_i, a_i$ .Math. $)^2)$, (31)

(74) where N is the number of tuples sampled from the experience replay buffer, y.sub.i is the target value, ϑ(s.sub.i, a.sub.1|θ) is the output value of the current critic, and this MSE loss function is a common supervised learning loss function that can reflect the fitting ability of the current critic and help the network better adapt to the target value y.sub.i.

(75) S**511**: Updating the Actor of the current network by calculating the sampled policy gradient, where Q(s,a|θ) represents the predicted values of the current critic for state s and action a, ∇.sub.aQ(s,a|θ) represents the action gradient on the predicted values, φ.sub.u(s|p) represents the action output of the current actor for state s, ∇.sub.uφ.sub.u(s|μ) represents the parameter gradient for the action output. The calculation formula for the gradient of this strategy is a Monte Carlo method that estimates the expected value using sampled tuple data.

(76) $\nabla_u J \approx \frac{1}{N}$ .Math.$_{i=1}^{N}$ $(\nabla_a Q(s, a$ .Math. $)$ .Math.$_{s=s_i, a=\ _u(s_i)}$ $, \nabla_u \ _u(s$ .Math. $)$ .Math.$_{s=s_i}$), (32)

(77) S**512**: if the current iterative times t can be divided by C, i.e. t % C=0, updating the network parameters, where θ′ and u′ respectively represent the parameters of the target critic and the target actor, and v is a hyperparameter less than 1 used for smoothly updating:

(78) $' = v \ + (1 - v) \ '$, (33) $u' = vu + (1 - v)u'$, (34)

(79) S**513**: t=t+1, repeating S**505** to S**512**.

(80) S**514**: If the current iterative times t is equal to n, that is, the maximum execution count has been reached, then episode=episode+1, then repeating S**504** to S**513** until the training process is complete and the optimal offloading strategy is obtained.

(81) It should be noted that the parts not elaborated in this description belong to the existing technology. Relevant technical personnel should understand that the above implementation is only intended to assist readers in understanding the principles and implementation methods of this disclosure, and the scope protected by this disclosure is not limited to such embodiments. Any equivalent substitution made on the basis of this disclosure is within the scope of protection of the rights of this disclosure.

Claims

1. A method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning, comprising the following steps: S**1**: constructing a vehicle system network, wherein the vehicle system network comprises Mobile Edge Computing (MEC) servers, Road Side Units (RSUs) and vehicular user sides connected in sequence; S**2**: constructing a task model for an application program, wherein tasks with temporal dependencies and data dependencies are modeled as a directed acyclic graph (DAG) task model; S**3**: constructing a task load model: calculating delay, energy consumption and incentive compensation for local computing, wherein a client-side vehicle processing tasks by itself, the client-side vehicle uploading the tasks to neighboring server-side vehicles, or the client-side vehicle uploading tasks to nearby RSUs based on the vehicle network system and the task model, respectively; S**4**: determining task priorities: first determining the priorities of each of subtasks according to allocation of predecessor nodes of the subtasks combining with dynamic network environment, and then scheduling according to a multi-queue algorithm; and S**5**: finding an optimal offloading strategy by using the deep reinforcement learning: each of the subtasks selects a corresponding execution device to complete the task offloading, so as to minimize energy consumption and paid incentive compensation under constraints of delay, and thereby improve quality of service for users; wherein in S**2**, constructing the task model for the application program comprises as follows: the tasks with the temporal dependencies and the data dependencies are modeled as the DAG task model, and the DAG task is modeled as G=(V,E), wherein a vertex set V={ϕ.sub.i|1≤i≤N}, N denotes a number of the subtasks in V; a triad is defined as ϕ.sub.i=(w.sub.i, d.sub.i, τ.sub.i) wherein w.sub.i, d.sub.i, τ.sub.i denote a computational workload, a data size, and a maximum completion time tolerance delay of the tasks ϕ.sub.i, respectively; E denotes a set of edges, and each of the edges e.sub.ϕ.sub.i.sub.,ϕ.sub.j∈E is a directed edge denoting the priorities of execution between the edges, indicating that the subtasks ϕ.sub.i need to be completed before ϕ.sub.j, and output of ϕ.sub.i is used as a part of input of ϕ.sub.j; if an edge exists from ϕ.sub.i to ϕ.sub.j, ϕ.sub.i is call as immediate predecessors of ϕ.sub.j; otherwise, ϕ.sub.j is called immediate successors of ϕ.sub.i; all the predecessors need to be completed before successors; a subtask without any predecessor node is a start node, also known as a source task, denoted as ϕ.sub.entry; a subtask without any successor node is an exit node, denoted as ϕ.sub.exit; by the DAG task model, the temporal dependencies and data dependencies between the tasks are clearly represented, and a basis for task scheduling is provide.

2. The method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning according to claim 1, wherein S**3** comprises the following steps: S**301**: interdependent subtasks executed at different vehicle nodes require data transfer; S**302**: calculating time for executing the tasks locally; S**303**: calculating the energy consumption for executing the tasks locally; S**304**: time t.sub.i, energy consumption e.sub.i and cost c.sub.i to be paid for executing a subtask ϕ.sub.i; and S**305**: creating an optimization problem considering the energy consumption and the incentive compensation for completing the tasks, so as to make an optimal offloading decision.

3. The method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning according to claim 2, wherein S**301** comprises as follows: there is some time to transfer computation results of the subtask ϕ.sub.j executed on the vehicle node m to subtask ϕ.sub.i executed on the vehicle node n, and the data transfer time is defined as follows:

$$TT_{j,i}(m,n)=\begin{cases}\frac{e_{j,i}}{r_{mn}}, & e_{mn}\in E^{RSU_i}\\ \min(\frac{e_{j,i}}{r_{mk}}+\frac{e_{j,i}}{r_{kn}}), & e_{mk}\in E^{RSU_i}\ \text{and}\ e_{kn}\in E^{RSU_i}\ \text{and}\ e_{mn}\ .\text{Math.}\ E^{RSU_i}\end{cases}, \quad (1)$$

wherein k, m and n represent labeling of the vehicle nodes respectively, ϕ.sub.j is the predecessor node of subtask of ϕ.sub.i, r.sub.mn represents a transmission bandwidth between the vehicle node m to the vehicle node n, e.sub.ϕ.sub.j.sub.ϕ.sub.i denotes a data size of dependent data of the subtasks ϕ.sub.i to ϕ.sub.j, e.sub.mn denotes whether or not the vehicle node m and the vehicle node n are within a communication range, and E.sup.RSU.sup.i denotes the set of the edges established by the vehicles through single-hop V2V communication within the communication range of a road side unit RSU.sub.i.

4. The method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning according to claim 2, wherein S**302** comprises as follows: for a case of executing the tasks locally, a completion time of the subtask ϕ.sub.i comprises a waiting time t.sub.i.sup.local,wait and a computation time t.sub.i.sup.local,compution; $t_i^{local,complete}=t_i^{local,wait}+t_i^{local,computation}$, (2) $t_i^{local,computation}=\frac{w_i}{f_c}$, (3) $t_i^{local,wait}=\max\{\text{avail}(V_c),\max(TT_{j,i}(m,V_c))\}$, (4) wherein V.sub.c denotes a client-side vehicle, avail(V.sub.c) denotes a time when the client-side vehicle finishes all the scheduled tasks, w.sub.i denotes an amount of computation required by the subtask ϕ.sub.i, and f.sub.c denotes a computational power of the client-side vehicle; for a case of executing the tasks on RSU.sub.j the completion time of subtask ϕ.sub.i comprises an upload time t.sub.i.sup.RSU.sup.j.sup.,upload, a waiting time t.sub.i.sup.RSU.sup.j.sup.,wait and a computation time t.sub.i.sup.RSU.sup.j.sup.,compution; $t_i^{RSU_j,complete}=t_i^{RSU_j,upload}+t_i^{RSU_j,wait}+t_i^{RSU_j,computation}$, (5) $t_i^{RSU_j,upload}=\frac{d_i}{r_{V_c,RSU_j}}$, (6) $t_i^{RSU_j,wait}=\max\{\text{avail}(RSU_j),\max(TT_{j,i}(m,RSU_j))\}$, (7) $t_i^{RSU_j,computation}=\frac{w_i}{f_{RSU_j}}$, (8) wherein r.sub.V.sub.c.sub.,RSU.sub.j denotes a transmission rate between the client-side vehicle and the specified RSU.sub.j, d.sub.i denotes the data size of the subtask ϕ.sub.i, and f.sub.RSU.sub.j denotes a computational power of the road side unit RSU.sub.j; the time to execute tasks on the server-side vehicles comprises a transmission time t.sub.i.sup.V.sup.k.sup.,trans, a waiting time t.sub.i.sup.V.sup.k.sup.,wait and a computation time t.sub.i.sup.V.sup.k.sup.,compution: $t_i^{V_k,complete}=t_i^{V_k,trans}+t_i^{V_k,wait}+t_i^{V_k,computation}$, (9) $t_i^{V_k,trans}=\frac{d_i}{r_{V_c,V_k}}$, (10) $t_i^{V_k,wait}=\max\{\text{avail}(V_k),\max(TT_{j,i}(m,V_k))\}$, (11) $t_i^{V_k,computation}=\frac{w_i}{f_k}$, (12) wherein r.sub.V.sub.c.sub.,V.sub.k denotes a transmission rate between a client-side vehicle V.sub.c to a specified server-side vehicle V.sub.k, and f.sub.k denotes a computational power of the server-side vehicle V.sub.k.

5. The method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning according to claim 4, wherein S**303** comprises as follows: $e_i^{local}=(t_i^{local,computation})^{-2}(w_i)^3$, (13) wherein ξ is a coefficient related to an architecture of a computer chip; calculating the energy consumption for executing the tasks at the RSUs: $e_i^{RSU_j}=p_{V_c}t_i^{RSU_j,upload}$. (14) wherein p denotes an upload power, so the incentive compensation to be paid to the road side unit RSU.sub.j for processing the subtask ϕ.sub.i is as follows: $C_{RSU_j,i}=p_j t_i^{RSU_j,computation}$, (15) wherein p.sub.j denotes a fee charged by the road side unit RSU.sub.j per unit time (s); calculating the energy consumption for executing the tasks on the server-side vehicles: $e_i^{V_k}=p_{V_c}t_i^{V_k,trans}$, (16) wherein the incentive compensation to be paid to the server-side vehicle k for processing the subtask ϕ.sub.i is as follows: $C_{V_k,i}=w_{i\ k}$, (17) wherein ρ.sub.k denotes a fee charged by the server-side vehicle k per unit of resource.

6. The method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning according to claim 5, wherein in S**304**, the time t.sub.i, the energy consumption e.sub.i and the cost c.sub.i to be paid for executing the subtask ϕ.sub.i are:

$$t_i=x_i^{local}t_i^{local,complete}+x_i^{RSU_j}t_i^{RSU_j,complete}+x_i^{V_k}t_i^{V_k,complete}, \quad (18)$$

$$e_i=x_i^{local}e_i^{local}+x_i^{RSU_j}e_i^{RSU_j}+x_i^{V_k}e_i^{V_k}, \quad (19)$$

$c_i = x_i^{\mathrm{RSU}_j} C_{\mathrm{RSU}_j,\ i} + x_i^{V_k} C_{V_k,\ i}$, (20) wherein x.sub.i.sup.local, x.sub.i.sup.RSU.sup.j and x.sub.i.sup.V.sup.k denote offloading decisions of the subtasks, and values of x.sub.i.sup.local, x.sub.i.sup.RSU.sup.j and x.sub.i.sup.V.sup.k take all 0 or 1, and only one of the three take 1; a total incentive compensation cost, the incentive compensation to be paid to the client-side vehicle to complete all the tasks, is denoted as:

$C(X) = \text{.Math.}_{i=1}^{N} c_i$, (21) a total energy consumption cost, the energy consumption of the client-side vehicle to complete all the tasks, denoted as: $E(X) = \text{.Math.}_{i=1}^{N} e_i$, (22) wherein X denotes an offloading decision vector and N denotes a number of the tasks; these formulas describe a relationship between the task offloading decisions, the execution time, the energy consumption and the incentive compensation, and may be used to optimize the task offloading to minimize the total energy consumption and total incentive compensation.

7. The method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning according to claim 6, wherein in S**305**, the optimization problem considering the energy consumption and the incentive compensation for completing the tasks is created:

$$:\min_{X} (X) = C(X) + E(X) s.t. C1: x_i^{\text{local}} + x_i^{\mathrm{RSU}_j} + x_i^{V_k} = 1, x_i^{\text{local}} \in \{0,1\} x_i^{\mathrm{RSU}_j} \in \{0,1\} x_i^{V_k} \in \{0,1\}, C2: t_i \le \min(l_{c,k},\ _i), x_i^{V_k} = 1 C3: t_i \le \min($$

wherein C(X) denotes the total incentive compensation cost, E(X) denotes a total computational cost, C1 denotes that each of the subtasks has to be offloaded to a certain device, C2 and C3 denote that if the subtasks are offloaded to server-side vehicles or RSUs, the completion time cannot exceed maximum allowable time thresholds l.sub.c,k or l.sub.c,RSU.sub.j, and C4 denotes that a weighted sum of the energy consumption and the incentive compensation is 1, and both are positive numbers; the optimization problem is decomposed into two subproblems: task priority determination and offloading decisions; firstly, the task priority determination is to determine an execution order of each of the subtasks, so as to satisfy the constraints of the delay and task dependency; then, a task scheduling order is used for the offloading decisions, the deep reinforcement learning is used to decide whether each of the subtasks is executed locally computation, offloaded to the RSUs, or offloaded to the server-side vehicles to minimize the weighted sum of the energy consumption and the incentive compensation.

8. The method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning according to claim 7, wherein S**4** comprises the followings steps: S**401**: starting the subtasks, assuming that Rank.sup.D(ϕ.sub.ready)=0, determining the priority of the successor subtasks based on scheduling arrangement of the predecessor subtasks, determining a ranking of the existing subtasks ϕ.sub.i∈ϕ.sub.ready, and computing a value of dynamic down-ranking as follows: $\mathrm{Rank}^D(\ _i) = \max_{\ _j \in \mathrm{pred}(\ _i)} \{\mathrm{Rank}^D(\ _j) + \overline{TT}^{\ D}_{\ _j,\ _i} + \overline{CT}^{\ D}_{\ _i}$, (24) wherein pred(ϕ.sub.i) is a set of the immediate predecessor nodes of the subtask ϕ.sub.i, CT.sub.ϕ.sub.i.sup.D is a dynamic average compuation time, and is affected by the allocation of ϕ.sub.j∈pred(ϕ.sub.i), C.sup.VC(ϕ.sub.i) denotes a set of the candidate server-side vehicles or infrastructures executing the subtask ϕ.sub.i, and ϕ.sub.ready denotes a set of ready subtasks and is defined as follows: $(\overline{CT}_{\ _i})^D = \frac{\text{.Math.}_{m \in C^{\mathrm{VC}}(\ _i)} CT(\ _i, m)}{\text{.Math. } C^{\mathrm{VC}}(\ _i) \text{.Math.}}\ _i \in \ _{\mathrm{ready}}$, (25) TT.sub.ϕ.sub.j.sub.,ϕ.sub.i.sup.D represents a dynamic average transmission time, is also affected by the allocation of ϕ.sub.j∈pred(ϕ.sub.i), and is defined as follows: $(\overline{TT}^{\ -}_{\ _j,\ _i})^D = \frac{\text{.Math.}_{m \in C^{\mathrm{VC}}(\ _i)} \mathrm{TT}_{\ _j,\ _i}(n,m)}{\text{.Math. } C^{\mathrm{VC}}(\ _i) \text{.Math.}}\ _i \in \ _{\mathrm{ready}}\ _j \in \mathrm{pred}(\ _i)$; (26) S**402**: the task scheduling depends not only on the priority but also on the dependencies between the tasks; one subtask is executed only when all its predecessor subtasks are completed; therefore, it is necessary to maintain a queue of the ready subtasks, Q.sup.ready, and a queue of executing the subtasks, Q.sup.exe, along with a queue of computed subtasks, Q.sup.finished, and a set of all the subtasks in the application program, T.sup.task; initially, ϕ.sup.entry subtasks are placed into Q.sup.ready from the set T.sup.task, the tasks in Q.sup.ready are sorted in ascending order according to the subtask priority determination formula, a front element of the queue is removed to place the tasks into the queue Q.sup.exe of executing the subtasks, and if there are the completed tasks in the queue Q.sup.exe, the completed tasks are placed into the Q.sup.finished queue; this may generate new ready subtasks, and if all the predecessor node subtasks of the subtask ϕ.sub.i in the set T.sup.task are in the queue Q.sup.finished, the subtask ϕ.sub.i becomes the ready subtask, and is placed in the queue Q.sup.ready, and the above steps are repeated until the set T.sup.task is empty; in the whole process of the subtask priority determination, the ready subtasks are dynamically determined according to execution results of the predecessor subtasks, and then the priority of the ready subtasks is determined.

9. The method of optimizing dependent task offloading in an Internet of Vehicles using deep reinforcement learning according to claim 8, wherein S**5** comprises the following steps: S**501**: initializing an experience replay buffer and defining a maximum number of the execution and initial episodes; S**502**: initializing parameters μ land θ of current actor and current critic after initializing the experience replay buffer; S**503**: initializing a parameter μ′←μ of target actor and a parameter θ′←θ of target critic separately; S**504**: obtaining an initial environment state s.sub.t and preparing an initial random noise n.sub.t for action exploration; S**505**: obtaining an action a.sub.t from time t=1 until t=n based on output of an Actor-C network of a current network and the random noise n.sub.t: $a_t = \ (s_t) + n_t$, (29) wherein n represents a number of indivisible subtasks in the application program; S**506**: executing the action a.sub.t, obtaining reward r.sub.t, and changing the environment state to s.sub.t+1; S**507**: storing (s.sub.t, a.sub.t, r.sub.t, s.sub.t+1) in the experience replay buffer, and offline training by using historical data in the experience replay buffer to improve stability and robustness of the model; S**508**: sampling N tuples {(s.sub.t, a.sub.t, r.sub.t, s.sub.t+1)} from the experience replay buffer for training a target network and updating the current network; S**509**: calculating the target value for each of the tuples y.sub.t by using the target network, wherein y.sub.t is a value related to the current state s.sub.t and the action a.sub.t, y.sub.t comprises the current reward r.sub.t and a target predicted value of a next state s.sub.t+1, and the target value is calculated using a following formula: $y_t = r_t + \ (s_{t+1},\ (s_{t+1}))$, (30) wherein γ is a discount factor, ϑ.sub.θ′ is the target critic, φ.sub.μ′ is the target actor, and these network parameters are initialized in S**503**; S**510**: updating the parameter θ of the current critic by minimizing target loss L(θ), wherein L(θ) is calculated from a mean squared error (MSE) loss function as follows:

$L(\ ) = \frac{1}{N} \text{.Math.}_{i=1}^{N} (y_i - \ (s_i, a_i \text{.Math.}\ )^2)$, (31) wherein N is a number of the tuples sampled from the experience replay buffer, y.sub.i is the target value, and ϑ.sub.θ(s.sub.i, a.sub.i|θ) is an output value of the current critic; S**511**: updating the Actor network of the current network by calculating a sampled policy gradient, wherein Q(s, a|θ) represents predicted values of the current critic for the state s and the action a, ∇.sub.aQ(s, a|θ) represents an action gradient on the predicted values, φ.sub.u(s|μ) represents action output of the current actor network for the state s, ∇.sub.uφ.sub.u(s|μ) represents a parameter gradient for the action output, and a calculation formula for the strategy gradient is a Monte Carlo method, and estimates expected values using sampled tuple data:

$\nabla_u J \approx \frac{1}{N} \text{.Math.}_{i=1}^{N} (\nabla_a Q(s, a \text{.Math.}\ ) \text{.Math.}_{s=s_i, a=\ _u(s_i)} \text{.Math.} \nabla_{u\ _u} (s \text{.Math.}\ ) \text{.Math.}_{s=s_i}$; (32) S**512**: if current iterative times t are divided by C, that is, t % C=0, updating the network parameters, wherein θ′ and u′ respectively represent the parameter of the target critic and the parameter the target actor, and v is a hyperparameter less than 1 used for smoothly updating: $'\ = v\ + (1 - v)\ '$, (33)

$u' = \mathrm{vu} + (1 - v) u'$, (34) S**513**: t=t+1, repeating S**505** to S**512**; S**514**: if the current iterative times t is equal to n, that is, the maximum number of the execution is reached, episode=episode+1; then repeating S**504** to S**513** until the training process is complete, and obtaining an optimal offloading strategy.